

The Verifiable AI Ecosystem: Verification-by-Default for the AI Internet

SentinelOps

September 1, 2025

Abstract

Three years from now, the “AI internet” can feel boringly safe: users do not click new checkboxes; they simply benefit from a fabric where every agent action is mediated, every outbound byte is attestable, and guarantees are backed by proofs, not vibes. This paper proposes a shared agenda, reference architecture, standards track, and ecosystem timeline to make verification-by-default practical and adopted. We target online behavioral safety, information-flow control, provenance, attestable compute, and operational assurance. Our approach pairs local, online monitors with global security claims (e.g., non-interference with typed declassification), enforces complete mediation via minimal adapters, standardizes reproducible evidence, and favors composable layers over monoliths. We define testable **MUST/SHOULD** requirements, conformance kits, and measurable SLOs to enable independent verification and cross-vendor interoperability.

Contents

1 Vision & Purpose	3
2 Core Principles (and Anti-patterns)	3
3 Reference Architecture	5
4 Standards Track	8
5 Mechanized Semantics & Machine-Checked Proofs	10
6 Research Threads	11
7 Risks & Mitigations	12
8 Ecosystem Development Timeline: 2025 to 2027	14

9 Mapping the Space: Taxonomy & Interfaces	16
9.1 Assurance Dimensions	16
9.2 Mechanism Grid	17
9.3 Interfaces	19
9.4 Design Choices & Trade-offs	20
9.5 Open Problems	20
10 Role Map	20
11 Engagement Modes	22
12 Current Initiatives	24
12.1 Provable Runtime Control	24
12.2 Unified Permissions (Call/Read/Write/Grant)	24
12.3 Certificates & Provenance	24
12.4 Deterministic Egress	25
12.5 Conformance Testing	25
12.6 DX Tooling	25
13 Near-term (6–12 months)	26
13.1 Standards Drafts (Public RFCs)	26
13.2 Interop Events	26
13.3 External Evaluations	26
13.4 Marketplace Alpha	26
13.5 Evidence Loop	27
14 Why This Can Become the Field’s Reference	27
15 How to Get Involved: On-ramps & Programs	27
15.1 Programs	28
15.2 Starter Checklist	28
15.3 Ecosystem KPIs	29
15.4 Code of Practice	29

1 Vision & Purpose

By 2028, the “AI internet” is unremarkably safe. Users no longer consent to safety features one checkbox at a time; instead, safety is ambient and evidenced. Every regulated emission is accompanied by a certificate, each certificate is machine-verifiable, and every enforcement point can be replayed deterministically. The promise of trustworthy AI becomes a property of the infrastructure rather than a claim in a slide deck.

The purpose of this paper is to align research, engineering, and governance on a verification-by-default paradigm for AI agents and autonomous systems. We argue for a fabric that is open, interoperable, and practical to adopt across academia, industry, and the public sector. Our proposal treats runtime safety as a first-class engineering concern, places proofs alongside execution, and treats evidence as a product artifact rather than an afterthought.

We write for a deliberately broad audience whose cooperation is essential to make this transition durable: formal methods and systems researchers who can crystallize assumptions and theorems; runtime-verification engineers who can translate specifications into finite monitors; platform builders who can mediate effects without compromising performance; auditors and evaluators who can validate guarantees independently; and standards bodies and policymakers who can turn repeatable practice into common baselines.

The scope of the agenda is concrete and bounded. We focus on online behavioral safety, information-flow control with typed declassification, provenance and supply-chain integrity, attestable compute when it is strictly necessary, and the operational assurance required to keep systems dependable in practice. We do not try to prove all aspects of model behavior, nor do we claim to solve interpretability; instead, we prove and enforce the boundaries (i.e., permissions, flows, and emissions) where harms materialize and where guarantees can be made precise. This focus keeps the approach tractable while still moving the field meaningfully toward evidence-backed safety.

2 Core Principles (and Anti-patterns)

The path to a verifiable AI ecosystem is not a collection of slogans but a discipline that connects theory to operations. Over the past decade, efforts have faltered for familiar reasons: specifications that aimed to capture everything captured nothing; detectors promised assurance but delivered triage; and closed, vertically integrated stacks hid their mechanisms from scrutiny. In response, we articulate five principles that work together as a coherent practice. Each principle stands on its own, yet the real strength emerges when they are pursued as a set.

Proofs where they matter. Not every property deserves a proof, but the boundaries at which harm occurs do. We therefore concentrate formal obligations on the actions and flows that shape real-world risk: who may do what, under which rates or budgets, and when information may pass from high to low. By shrinking the target to the interface where effects happen, proofs become tractable and immediately useful. The *anti-pattern* here is the monolithic specification that aspires to totality; it stalls on day one, resists iteration, and diffuses accountability. Targeted, composable obligations let teams deploy, measure, and refine without waiting for perfection.

Runtime paired with proofs. Guarantees must survive contact with execution. Local, online monitors enforce prefix-closed policies during runtime, while global claims (such as non-interference with typed declassification) explain what acceptance actually means. This pairing delivers prevention and interpretation together: the monitor denies unsafe prefixes as they arise, and the proof establishes why acceptance implies the desired security property. The *anti-pattern* is reliance on detectors alone. Detectors can be valuable as defense-in-depth, but they cannot certify compliant behavior by construction, and they provide no principled bridge from an alert to a guarantee.

Complete mediation. Assurance collapses when effects slip past the model. Every tool invocation, filesystem access, network egress, and declassification must traverse audited, minimal adapters so that all observable behavior is representable on a labeled alphabet. Mediation is what makes runtime enforcement and replay possible; it also creates a stable surface for standards. The *anti-pattern* is the “trusted side-channel” or unmodeled effect that bypasses adapters and silently erodes the proof context. Once mediation is incomplete, guarantees degrade into anecdotes.

Open standards for evidence. Evidence should travel farther than the code that produced it. We therefore commit to portable certificate schemas, deterministic egress profiles, and permission models that any conforming stack can implement and any independent party can verify. Replays, not retrospectives, become the default response to incidents. The *anti-pattern* is a proprietary evidence format or an unverifiable claim that requires trust in a vendor’s internals. When evidence is machine-checkable and reproducible, assurance becomes a property of the ecosystem rather than a promise of any single platform.

Composability over monoliths. Progress should come from improving components, not replacing entire stacks. Specifications, monitors, labelers, runtimes, and attestations must remain swappable behind stable interfaces so that teams can compete on quality and performance without sacrificing interoperability. This modularity enables rapid iteration while preserving a shared assurance story. The *anti-pattern* is vertical integration that obscures mechanisms from auditors and forecloses interoperation, trading near-term velocity

for long-term fragility.

Taken together, these principles define a pragmatic posture: prove obligations that govern harm; enforce them online where decisions are made; ensure every effect is mediated; publish evidence in forms others can verify; and build the system as a set of replaceable parts. Adopted incrementally, they deliver immediate, measurable gains. Adopted together, they make the “boringly safe” AI internet not just imaginable, but routine.

3 Reference Architecture

The ecosystem we propose is intentionally layered. Rather than a single, vertically integrated stack, it is a composable sequence of roles that meet at clean interfaces and share a common theory of evidence. Figure 1 summarizes the arrangement. Layers A through C define the semantics of behavior and its enforcement at runtime; Layers D through F make those semantics durable as evidence, govern how assurance travels across organizations, and operationalize the system day to day. The dashed line in the figure marks the critical trust boundary between build-time artifacts and runtime mediation. Proofs and automata are produced above the line; decisions and certificates are produced below it. The promise of the architecture is that *what* you prove is *what* you run, and that every emission carries enough context to be verified independently.

A. Policy & Specs. The foundation is a set of action-level policies that specify who may act, which actions they may take, and when those actions are in force. These policies include rate and budget limits to shape behavior over time, as well as permission epochs that provide precise semantics for revocation and snapshotting. Information-flow control (IFC) rules attach labels to events, define low-view projections with erasure, and enumerate typed declassification sites that justify authorized releases. To connect online enforcement to global security claims, the specifications also encode unwinding obligations and local predicates such as **MonNI**, which is sufficient to imply the non-interference claim **NI** under complete mediation. In particular, it **MUST** be the case that specifications are prefix-closed and compilable to finite monitors; declassification rules **MUST** name principals, record justifications, and include time-to-live (TTL) bounds; and permission epochs **MUST** define snapshot semantics and revocation behavior precisely enough to preserve finiteness at the monitor.

B. Proofs & Synthesis. Given the specifications, a synthesis pipeline compiles them into deterministic finite automata and product monitors, accompanied by machine-checked lemmas that establish soundness and, where tractable, completeness. The artifacts are expressed in

mainstream proof assistants (e.g., Lean, Coq, Isabelle) so that theorems, obligations, and compilation correctness can be audited without relying on any particular vendor’s tooling. Because labels and permissions are enforced at field granularity, the pipeline also emits proof-carrying labelers: concise witnesses that attest to Merkle-path membership for selected content and to bounded taint derivations for Information Flow Control (IFC) joins. Diversity of implementation reduces correlated failure: at least two independent labelers should validate the same witness schema. To make artifacts align with shipped binaries, implementers **MUST** publish proof hashes and automata hashes and bind them to releases; they **MUST** also provide counterexample shrinking so that failing obligations can be diagnosed with minimal traces.

C. Mediated Runtime. At execution time, a small sidecar mediates every effect that could change the outside world or leak information: tool invocations, file reads and writes, declassification calls, and streaming egress. Enforcement follows a deny-wins policy, so that a single negative verdict prevents an unsafe effect from occurring. Because timing and length can themselves carry information, the runtime adheres to a deterministic egress profile with fixed chunk size, flush cadence, locale, and timezone, enabling faithfully reproducible replays. Adoption proceeds by clauses: systems begin in **observe** mode to learn and calibrate, shift to **shadow** to compare live decisions with proposed ones, and finally move to **enforce** when reject budgets and operator workflows are in place. Two invariants keep the evidence meaningful: no effect may occur without a corresponding event on the alphabet, and no field may appear in that event without an attached witness. Stated as requirements, there **MUST** be no effects without events and no fields without witnesses.

D. Evidence & Provenance. Every emission yields a certificate that binds the runtime’s decisions to the artifacts produced at build time. Each certificate includes the hashes of the governing policy, proof, automata, and labeler; the verdicts returned by local monitors; the result of permission decisions; and the outcomes of witness checks. The software supply chain is recorded alongside these emissions through signatures, software bills of materials (SBOMs), and verifiable build provenance, with optional entries in public transparency logs for broader auditability. To keep the assurance portable and durable, evidence **MUST** be machine-verifiable and archivable, and each release **MUST** ship with a replay kit that makes certificates executable objects rather than static records.

E. Optional Attestable Compute. Some settings require attesting not only to *what* was emitted but to *how* the code ran. In those cases, trusted execution environments (TEEs)

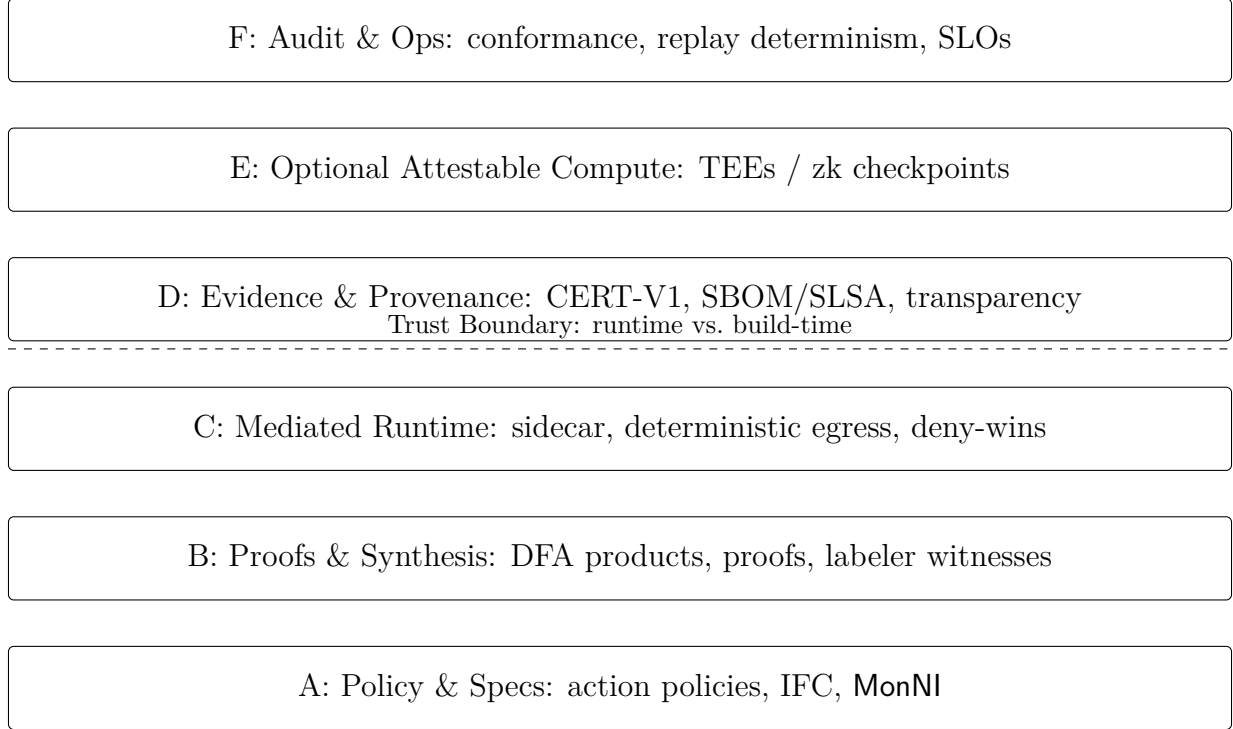


Figure 1: Layered architecture with a clear trust boundary between build-time artifacts and runtime enforcement.

or zero-knowledge (zk) proofs provide additional anchors for confidence: TEEs attest to enclave or Virtual Machine (VM) state using vendor-backed quotes; zk systems attest to the execution of specific computations via succinct proofs. These mechanisms are powerful but not free. They should be deployed selectively, and adopters should document the trust and cost envelopes explicitly so that auditors can weigh the benefits against residual risks and performance budgets.

F. Audit & Ops. Finally, assurance becomes a practice. Operations teams rely on deterministic replay for incident response, using minimal counterexamples to localize faults and dashboards to track fleet-wide health. Formal conformance suites provide a neutral yardstick for implementations, and service-level objectives (SLOs) translate promises into measurable targets: certificate verification rates, replay determinism, adherence to reject budgets, and end-to-end monitor latency. When these targets are published and met consistently, safety ceases to be a claim about intentions and becomes a property of day-two operations.

4 Standards Track

Standards turn a philosophy into artifacts that many parties can implement and audit. Our track is intentionally narrow and testable: each profile specifies exactly what evidence must be produced at runtime, how emissions are shaped for reproducibility, how permissions are decided, which preconditions make local monitoring sufficient for global claims, and how conformance is demonstrated. The goal is portability across vendors and durability across time without privileging any single stack.

CERT-V1: Certificate Schema (per emission). CERT-V1 defines the record that travels with every regulated emission. A compliant certificate **MUST** include cryptographic bindings to the build-time artifacts (i.e., `policy_hash`, `proof_hash`, `automata_hash`, and `labeler_hash`) and it **MUST** record the runtime decisions that justify the emission, namely the `ni_monitor` verdict (taking one of {`inapplicable`, `accept`, `reject`, `error`}) and the `permit_decision` (`accept`, `reject`, or `error`). To make the label story checkable, the certificate **MUST** carry booleans for `path_witness_ok` and `label_derivation_ok`. Finally, to bind evidence to a concrete deployment, it **MUST** record the current `epoch`, a `bundle_id`, and the `sidecar_build`. Where additional trust anchors are available, the record **SHOULD** include an attestation (TEE quote or zk digest) and a `transparency_ref` suitable for inclusion in an append-only log. With these fields present, a third party can verify the emission without access to vendor internals.

Example.

```
{
  "policy_hash": "sha256:...",
  "proof_hash": "sha256:...",
  "automata_hash": "sha256:...",
  "labeler_hash": "sha256:...",
  "ni_monitor": "accept",
  "permit_decision": "accept",
  "path_witness_ok": true,
  "label_derivation_ok": true,
  "epoch": 42,
  "bundle_id": "pab-2025.09.01+build17",
  "sidecar_build": "1.3.0",
  "attestation": null,
  "transparency_ref": "rekor:logindex/123456"
}
```

EGRESS-DET-P1: Deterministic Egress Profile. Assurance depends on replays that behave like the original run. EGRESS-DET-P1 therefore fixes the degrees of freedom that leak through timing and length channels. A conforming runtime **MUST** emit under a declared profile that fixes `-chunk-bytes` and `-flush-ms`, and it **MUST** pin locale and timezone so that textual and temporal formatting round-trip. It **MUST** also declare its padding policy, making observable lengths predictable under replay. To support risk assessment, the profile **SHOULD** publish an empirical capacity estimate (in bits per second) and a measured false-block rate under typical loads. These declarations do not eliminate all residual channels, but they make the remaining surface measurable and amenable to independent calibration.

PERM-UNIFY-R1: Unified Permissions. Modern agents act across tools and documents; the permission story must be coherent across both. PERM-UNIFY-R1 stipulates that a single decider **MUST** govern `Call`, `Read`, `Write`, and `Grant` operations, with ABAC/ReBAC guards expressed over principals, resources, and contexts. Because decisions often depend on specific fields, the decider **MUST** accept field-level witnesses so that membership proofs and taint derivations are verified rather than assumed. To keep monitors finite, the model **MUST** include permission epochs whose semantics support safe revocation and snapshotting. For operational clarity, the decider **SHOULD** produce concise explanation strings suitable for CI logs and audits so that denials can be triaged without spelunking through code.

MON-NI-BRIDGE: Preconditions for Local $\rightarrow$ Global NI. Local enforcement buys little unless it lifts to a global claim. MON-NI-BRIDGE states the preconditions under which acceptance by an online predicate such as `MonNI` implies termination-insensitive non-interference. Concretely, a compliant deployment **MUST** provide complete mediation of effects, it **MUST** satisfy the standard unwinding checks for the chosen NI variant, it **MUST** demonstrate that `MonNI` accepts all prefixes observed at runtime, and it **MUST** adhere to typed declassification rules expressed in the specification. Under these conditions, low-view projections of traces remain equivalent across secret variants, establishing TI-NI as the outcome rather than an aspiration.

TRACE-REPLAY-KIT: Canonical Replays & Conformance. Claims are meaningful when others can reproduce them. TRACE-REPLAY-KIT defines the bundle that turns certificates and traces into an oracle. A distribution that targets conformance **MUST** ship reproducible traces with pinned fixtures and environment configuration, including the egress profile parameters; it **MUST** include pass/fail oracles that encode the expected monitor and permission outcomes; and it **MUST** support counterexample shrinking so that failing

behaviors are reduced to minimal, actionable traces. With this kit in hand, independent evaluators can verify correctness, measure drift, and compare implementations.

Together, these profiles separate mechanism from assurance: CERT-V1 tells us *what* was decided and *why*; EGRESS-DET-P1 ensures those decisions can be replayed; PERM-UNIFY-R1 keeps authorization coherent across surfaces; MON-NI-BRIDGE explains when online acceptance is enough for a global claim; and TRACE-REPLAY-KIT turns the whole story into a test that anyone can run. The result is an ecosystem where evidence is portable, failures are diagnosable, and improvements are measurable.

5 Mechanized Semantics & Machine-Checked Proofs

Our assurance claims are intentionally modest but precise: (i) the monitors we ship accept exactly the traces our specifications denote; (ii) under complete mediation and typed declassification, online acceptance by **MonNI** suffices for termination-insensitive non-interference; and (iii) the deterministic egress profile makes replays faithful at the level that matters for audits. Because prose can drift while code evolves, we accompany the standards in §4 with a compact Lean artifact that turns each claim into a checkable theorem.

Scope and design. The Lean project defines a labeled alphabet and trace semantics, a low-view projection with erasure, and a typed-declassification judgment. It encodes **MonNI** as a prefix-wise predicate and formalizes our ActionDSL fragment for authorization, rate shaping, and IFC. Compilation is modeled as a translation from ActionDSL to a deterministic finite automaton (DFA) over the labeled alphabet. The point is not to cover every conceivable clause, but to mechanize the subset we standardize and to keep the bridge from specification to runtime small, readable, and auditable.

Obligations proved. We state and prove three obligations that line up with the fields in CERT-V1 and the parameters in EGRESS-DET-P1. First, *Compilation Soundness*: for the supported fragment, the language of the emitted DFA equals the denotation of the specification. Second, *Local $\Rightarrow$ Global Confidentiality*: if **MonNI** accepts every observed prefix and the deployment satisfies complete mediation and typed declassification, then low views are equivalent across secret variants (TI-NI). Third, *Replay Faithfulness*: given fixed chunk size, flush cadence, locale, and timezone, the replay function is observationally equivalent on low views to the original run. These theorems appear in Appendix H as H.1–H.3.

Making proofs operational. To keep proofs tied to what actually runs, the build publishes two hashes: `proof_hash` for the Lean artifact and `automata_hash` for the DFA JSON emitted by the compiler. Both hashes appear in CERT-V1, so auditors can confirm that the monitor in production is the one the proof speaks about. Continuous integration pins Lean and dependencies, builds the olean caches, regenerates theorem docs, and fails the pipeline if any theorem changes its statement or if the emitted hashes diverge from those recorded in certificates. In short, the Lean module is not a paper appendix, it is part of the product, and its artifacts travel with the evidence we expect others to verify.

6 Research Threads

Progress requires focus. We therefore pursue five tightly scoped research threads that connect formal advances to runtime practice and define unambiguous measurements for success. Each thread yields artifacts that can be implemented, audited, and, crucially, benchmarked in the open.

Beyond TI-NI. Termination-insensitive non-interference (TI-NI) is a practical starting point, but real deployments also face termination-sensitive behaviors and probabilistic flows. Our aim is to extend the local-to-global bridge so that online acceptance implies stronger guarantees without abandoning decidability. Concretely, we will characterize termination-sensitive NI variants and probabilistic flow models, derive unwinding obligations that remain monitorable, and publish machine-checked proofs alongside runnable predicates. The immediate outcome is a formal bridge with runtime-checkable preconditions; the longer-term benefit is a path to stronger confidentiality claims under realistic workloads.

Revocation & Epochs. Revocation is essential in dynamic environments, yet naive models explode monitor state. We treat *epochs* as the organizing device: permissions evolve through explicit snapshots whose rollover rules preserve finiteness. The research task is to prove that these semantics keep products minimal while preserving safety, and to ship reference monitors that implement them faithfully. Success is measured operationally: the *time-to-effect (TTF) of revocation* remains bounded, with $\text{TTF-revocation} \leq N$ events under load, and the compiled monitors remain finite and amenable to minimization.

Field-level IFC at Scale. Information-flow control delivers value only if field-level claims verify quickly at runtime. We develop label algebras that compose cleanly across adapters, and we compress witnesses so membership proofs and bounded taint derivations remain

small. Dual implementations reduce correlated failure and make verifier diversity routine. The bar for success is explicit: witness verification completes in under a millisecond at the 99th percentile, with sustained throughput on production traces and no loss of soundness.

Interop with zk/TEE. Some contexts require attesting *how* code ran. We explore two complementary routes. First, we design partial proofs for streaming prefixes, incremental or folded zk arguments that amortize proving cost per kilobyte with periodic checkpoints. Second, we specify side-channel constraints and verification SLOs for TEEs so quotes can be treated as first-class evidence. The outcomes are quantitative: amortized proving cost/KB and end-to-end latency within declared envelopes, plus published SLOs for attestation verification in the presence of transient failures.

Residual Channels under Deterministic Egress. Deterministic egress profiles bound, but do not eliminate, covert channels. We combine formal channel models with empirical calibration on live agents to estimate residual capacity and operational costs. The artifacts are reproducible harnesses and public curves: capacity bounds in bits per second under specified profiles, together with false-block trade-offs that help operators set padding policies responsibly.

Taken together, these threads move the stack from plausible arguments to measurable guarantees. Each delivers proofs and runnable components; each publishes targets and datasets so progress is comparable over time; and each tightens the link between what we can claim in theory and what we can enforce online.

7 Risks & Mitigations

A credible roadmap acknowledges the ways systems fail in practice and specifies how those failures are detected, contained, and learned from. The following risks recur across deployments; each is paired with concrete mitigations that can be implemented today and audited tomorrow.

Labeling errors. Information-flow labels and permission annotations are only as good as the evidence that backs them. When labels drift or are misapplied, monitors make the wrong decisions with high confidence. We therefore adopt proof-carrying labelers that attach concise witnesses, Merkle-path membership for selected content and bounded taint derivations for joins, to every field that matters. To reduce correlated failure, we maintain dual implementations that validate the same witness schema and fail closed when they

disagree. In continuous integration, we run adversarial corpora that include polyglot formats and content-smuggling attempts to exercise edge cases and expand coverage. Together, these practices turn labeling from a convention into a checkable claim.

Scheduler variance. Runtime behavior changes under load, and scheduling artifacts can masquerade as policy violations or conceal real ones. We counter this with documented execution models (single-queue or two-queue) and with a deny-wins policy that ensures any negative verdict blocks the effect regardless of timing. Deterministic egress further removes clock-dependent variance from the observable surface. Finally, replay determinism tests are part of every release, so regressions in scheduling semantics are caught before production and incidents can be reproduced byte-for-byte when they occur.

Fragmentation. Ecosystems fracture when every team invents its own evidence format and monitoring surface. To avoid that outcome, we publish open Request for Comments (RFCs) that specify narrow, testable standards; we host quarterly interoperability events where independent stacks cross-verify certificates and replays; and we establish multi-stakeholder governance so deprecations and migrations are transparent. Sector-specific reference profiles provide a common baseline for regulated domains without freezing innovation at the implementation layer.

Performance regressions. Safety that slows systems to a crawl will be bypassed. We design for predictable overhead by using amortized $O(1)$ window checks for rate limits and by minimizing product automata so monitors do not carry unnecessary state. Where hot paths dominate, we generate specialized code from the reference DFA and guard it with equivalence tests to the canonical semantics. Performance budgets are enforced in CI with threshold gates on p95 latency and throughput, turning regressions into failing builds rather than unpleasant surprises in production.

In sum, these mitigations make failure instructive instead of catastrophic: labels are evidenced and redundantly verified; scheduling variance is modeled and replayed; the ecosystem converges on shared, testable interfaces; and performance remains within explicit budgets. The result is an operational posture where risks are anticipated, measured, and continuously reduced.

8 Ecosystem Development Timeline: 2025 to 2027

Safety emerges as a cadence (i.e., *spec* $\rightarrow$ *prove* $\rightarrow$ *run* $\rightarrow$ *demonstrate* $\rightarrow$ *standardize*) repeated across domains until it becomes routine. The three-year plan that follows treats each year as a concrete stage with public artifacts, measurable exit criteria, and explicit dependencies.

2025: Prove It Works (multi-site pilots)

The priority in 2025 is to demonstrate end-to-end viability across more than one setting. We will publish a reference stack that carries a policy all the way to execution and back to evidence: specifications compile to DFA/product monitors; a small sidecar mediates effects; each emission produces a certificate; and a replay harness makes runs reproducible in third-party hands. The repository will include documentation, example adapters, and traces that exercise common patterns. Unified permissions with field-level witnesses are part of the baseline, and adoption proceeds clause by clause, first in **observe** to learn, then in **shadow** to compare decisions, and finally in **enforce** when operators are ready.

To declare the year a success, we set concrete exit criteria. Policy coverage must reach at least 90% of mediated effects across pilots. Certificate verification must succeed at least 99.99% of the time when checked by an independent verifier. Replay determinism must reach 99.9% under EGRESS-DET-P1 profiles. Online monitoring must stay within a p95 overhead of < 2 ms per event. And there must be no unmediated egress incidents; if any occur, they are documented with public post-mortems and replay bundles. The principal risks are practical: partners must be available to run pilots; privacy and legal reviews must permit public artifacts; and engineering bandwidth must cover witness generation across a diversity of adapters.

2026: Prove It Scales (standards + marketplace)

Having shown viability, 2026 focuses on scale and interoperability. A FRO-style consortium will publish variants of NI, epoch semantics for safe revocation, and techniques for monitor minimization, accompanied by joint proofs and reference monitors. In parallel, a marketplace alpha will list verified agents and templates and will trigger automated recertification when dependencies change. Independent evaluations by runtime-verification experts and red teams will be commissioned and published, and interoperability demos will show certificates flowing through transparency logs with optional zk/TEE anchors.

Our exit criteria reflect ecosystem health rather than the success of a single codebase.

At least 10 verified agents or templates should be listed with recertification running on updates. At least 5 independent evaluations should be published, and at least 3 interop events should produce public results. New adopters should be able to move from **observe** to first **enforce** within a day, demonstrating that the tooling and guidance are mature. The main dependencies are procedural and social: standardization requires time and attention, CERT-V1 must avoid premature complexity, and marketplace governance must set clear policies for moderation, revocation, and evidence quality.

2027: Verification-by-Default (platform adoption)

By 2027 the center of gravity shifts to platforms. Major providers integrate certificate gates into their deployment paths for autonomous agents so that unverified emissions cannot reach production by accident. A position paper and a set of ecosystem standards, authored across organizations, consolidate the technical foundations and document stable interfaces. Reference implementations appear in multiple languages to lower barriers to entry, and sector-specific profiles, for healthcare, finance, and the public sector, translate the standards into domain-ready defaults.

We declare success when verification becomes a default stance rather than an optional add-on. At least 5 regulated deployments should operate behind certificate gates, and at least 3 platforms should include certificate verification as a built-in capability. Operationally, incident MTTR should fall to ≤ 2 hours when replay kits are used, and public evidence packs should make audits feasible without vendor access. The remaining risks are largely about timing and fit: platform roadmaps must align with the gates, and the gates themselves must be fast and developer-friendly so that safety improves the experience rather than obstructing it.

Cross-cutting SLOs & Governance

Across all three years we publish and uphold service-level objectives so progress is visible and falsifiable. Certificate verification rates remain at 99.99% and replay determinism at 99.9%, with monthly reporting and drift analysis. Reject budgets preserve business continuity by design; clause-level enforcement and safe fallbacks let teams raise assurance without wholesale rewrites. Governance remains open and predictable: numbered RFCs invite participation, quarterly interop events keep implementations honest, and deprecations follow transparent policies with migration guides. In combination, these practices turn the roadmap into a habit, the cadence that makes safety evidence-backed and, ultimately, boring.

9 Mapping the Space: Taxonomy & Interfaces

The public conversation about safety is noisy: prompts and heuristics dominate headlines, while guarantees remain elusive. What scales across organizations is not bespoke policy prose but *evidence*, properties that can be proved once and enforced online many times. This section maps the territory in which proofs matter, explains how layers compose without leakage, and identifies the interfaces that let independently built stacks interoperate without sacrificing assurance.

9.1 Assurance Dimensions

Behavioral Safety. We treat behavior as membership in a prefix-closed language over a labeled alphabet. Policies compile to finite monitors that decide, per event or byte, whether execution remains within that language; a deny-wins rule ensures that any negative verdict prevents an unsafe effect. Rate limits are enforced with amortized $O(1)$ windows so cost remains predictable at scale. Deterministic semantics make runs reproducible, which in turn makes incidents diagnosable: a trace either belongs to the language, or it does not.

Information Flow (IFC). Where confidentiality matters, events carry labels and low views are formed by projection with erasure. Typed declassification specifies the narrow circumstances under which high information may flow to low. Non-interference (NI) provides the global claim (i.e., low views are indistinguishable across secret variants) while a local predicate such as **MonNI** supplies sufficient online conditions for that claim under complete mediation. In practice, this bridge lets operators reason about flows one prefix at a time without abandoning end-to-end guarantees.

Provenance. Assurance decays when artifacts cannot be traced. We therefore bind every build and every emission to cryptographic evidence: signed artifacts, SBOM/SLSA provenance for the build path, and per-emission certificates that link runtime verdicts to the governing proof and automata hashes. When feasible, certificate digests enter append-only transparency logs so that inclusion proofs are auditable by anyone with the log.

Compute Attestation. Sometimes it is not enough to know *what* was emitted; one must attest to *how* the computation ran. Trusted execution environments (TEEs) offer hardware-backed quotes over enclave or VM state, and zero-knowledge (zk) proofs offer succinct attestations for specific computations. Both are powerful; both come with costs. We use them selectively and record the trust and performance envelopes explicitly.

Operations. Evidence becomes operational when it supports routine practice: replays are deterministic, counterexamples are minimal, and conformance suites let third parties validate a stack end to end. The discipline here is to treat reproducibility and comparability as first-class SLOs, not as best-effort conveniences.

9.2 Mechanism Grid

To anchor trade-offs in concrete choices, Table [9.2](#) summarizes the mechanisms we employ, why they outperform common alternatives, and the measurements that keep them honest.

Problem	Mechanism	Why this, not that	What to measure
Enforce per-event policy (auth/rates/shape)	DFAs over a labeled alphabet; product automata; deny-wins	Deterministic, auditable; aligns with runtime verification theory	p95 monitor latency; false negatives=0; window invariants honored
Guarantee confidentiality with declass	IFC + NI variants; projection/erasure; unwinding; local-to-global via MonNI	Global NI is non-online; MonNI gives a decidable local predicate	Prefix acceptance rate; equality of low views in replays
Prove tool/doc access semantics	Unified permissions (Call/Read/Write/Grant) with ABAC/ReBAC; epochs	One calculus for tools and documents; epochs keep monitors finite	Time-to-revocation; epoch rollover safety; decision explainability
Bind outputs to proofs	Per-emission certificates + transparency log	Portable verifiability; public append-only evidence	% certs independently verified; inclusion-proof latency
Build integrity	SLSA + in-toto provenance; DSSE signatures	Industry-standard supply-chain controls	SLSA level; provenance completeness; key rotation hygiene
Compute attestation	TEE quotes or zk proof digests	TEEs pragmatic; zk stronger but costlier, use selectively	Quote verification rate; zk proving cost/latency envelopes

Table 9.2: Mechanisms, rationale, and the metrics that discipline them.

9.3 Interfaces

Assurance survives organizational boundaries when interfaces are explicit and artifacts travel well. We therefore define contracts that make each handoff checkable.

Spec $\rightarrow$ Monitor. A specification arrives as an action-level document governing authorization, rate shaping, IFC with typed declassification, and permission epochs. A compiler returns a deterministic automaton (serialized as DFA JSON with states, transition function δ , and accepting set), together with hashes that bind the automaton to the original specification and lemmas that establish compilation soundness. The contract is simple to state and easy to test: $L(\text{automaton})$ must equal spec, and both sides publish their hashes in release artifacts so mismatches are visible.

Runtime $\rightarrow$ Evidence. At execution time, the runtime emits CERT-V1 records (Section 4) that capture the relevant hashes, the monitor verdicts, permission decisions, and the outcomes of witness checks. Optional attestations (TEE quotes or zk digests) are embedded as fields rather than attachments, and certificate digests may be logged to transparency systems so auditors can verify inclusion without privileged access.

Adapters (Frozen Effects). Effects such as HTTP GET and file read are mediated by adapters whose behavior is fixed and whose witness semantics are explicit. For content selection, witnesses provide Merkle-path membership; for IFC joins, witnesses prove bounded taint derivations. The rule is categorical: no side effects occur without an event on the alphabet, and no fields appear without a membership proof.

Attestation Hooks. When attestation is required, the evidence includes a reference to the TEE quote and its collateral or to the zk proof digest along with circuit and version identifiers. Verifiers treat these references as inputs to independent checks rather than as opaque blobs.

Replay Bundles. To make evidence executable, releases include canonical traces with state deltas, pinned fixtures, and declared environment parameters (locale, timezone, chunk and flush settings). Certificates index the traces, turning “replay determinism” into a measurable SLO instead of an aspiration.

9.4 Design Choices & Trade-offs

Adopting deterministic automata over ad-hoc code favors properties we can reason about and minimize; it also enables shrinkable counterexamples when obligations fail. Starting with termination-insensitive NI acknowledges the realities of online checking while leaving a clear path to stronger guarantees as monitoring predicates evolve. TEEs and zk attestations are complementary: TEEs deliver practical, system-level anchors with operational trust assumptions; zk systems deliver cryptographic assurances for specific computations at higher cost. Authorization PDPs in the style of OPA remain valuable at service edges, but event-level monitors are required to enforce byte- and field-level obligations in agents with streaming I/O. Finally, pairing SLSA/in-toto provenance with transparency logs moves the supply-chain story from vendor claims to ecosystem evidence.

9.5 Open Problems

Even with deterministic egress, residual channels remain; bounding their capacity rigorously and calibrating them empirically on live agents is an open and valuable task. Field-level IFC at cloud scale raises practical concerns about witness size and verifier throughput, motivating compression schemes and verifier diversity that preserve soundness without sacrificing latency. Partial proofs for streams promise amortized zk costs but require careful circuit design and checkpointing strategies. Cross-organizational declassification needs governance models that honor privacy while producing auditable policies. And product automata, while precise, invite state blow-up; principled minimization and hot-path specialization must keep monitors small without drifting from the reference semantics.

In aggregate, this taxonomy and its interfaces transform safety from a local property of a codebase into a portable property of evidence. Layers stay swappable, proofs travel with artifacts, and the contracts are tight enough that independent verifiers can reproduce conclusions without access to internals. That is the essence of an ecosystem designed to be both open and dependable.

10 Role Map

A verifiable AI ecosystem is a coalition project. Progress depends on different institutions doing what they are best at and publishing artifacts that others can build on. The roles below describe capabilities that already exist in the community and the specific contributions that convert intent into motion. A single organization may wear more than one hat; what matters is that the outputs are public, testable, and timed to the cadence set out in the

roadmap.

Funders & Conveners. Philanthropies, public agencies, and mission-driven investors bring capital, convening power, and program design experience. Their leverage lies in setting expectations for open artifacts and in underwriting the unglamorous glue, interoperability events, shared testbeds, and long-lived infrastructure. We ask funders to back focused research organizations (FROs) that bridge theory and runtime, to finance interop events where independent stacks cross-verify evidence, and to require public evidence packs (certificates, replays, provenance) as deliverables in grants and challenge programs. By attaching dollars to evidence rather than demos, funders accelerate the norms this ecosystem needs.

Research Labs. Academic groups and industrial research teams contribute proofs, algorithms, and testbeds that make guarantees precise. Their comparative advantage is to turn assumptions into theorems and to mechanize those theorems in mainstream proof assistants. We ask labs to co-author reference proofs for compilation soundness and monitor acceptance, to formalize NI variants and epoch semantics that keep monitors finite, and to run adversarial generators that expose edge cases in labeling and egress. When proofs travel with runnable predicates, deployments inherit rigor without depending on a single implementation.

Builders & Platforms. Platform teams and product groups turn specifications into mediated runtimes, adapters, certificate services, and developer tooling. They determine whether guarantees survive contact with scale. We ask builders to adopt the shared standards (i.e., CERT-V1 for certificates, EGRESS-DET-P1 for deterministic egress, and PERM-UNIFY-R1 for unified permissions) to export replay bundles by default, and to support clause-level rollouts that move from **observe** to **shadow** to **enforce** safely. Shipping these capabilities behind stable interfaces lets operators prove compliance without trusting opaque internals.

Auditors & Evaluators. Independent assessors, runtime-verification firms, and red teams provide the external pressure that keeps claims honest. Their value is to measure coverage, residual risk, and drift with methods others can reproduce. We ask evaluators to curate conformance suites that exercise common and adversarial traces, to publish public residual-risk reports that quantify accept/deny precision and monitor overhead, and to monitor transparency logs for certificate inclusion and revocation. When audits consume the same evidence that production emits, assurance scales beyond any single vendor.

Standards & Policy. Standards bodies and policy groups supply process, legitimacy, and alignment with regulatory realities. They turn repeated practice into shared baselines. We

ask these institutions to prioritize narrowly scoped, testable standards whose **MUST** and **SHOULD** clauses can be checked mechanically, and to publish sector-specific reference profiles that translate the core standards into domain-ready defaults. Clear deprecation policies and migration guides keep the ecosystem moving without stranding adopters.

Communities. Open-source maintainers, meetups, and practitioner networks create the continuity that bureaucracies and product cycles often lack. They keep the lights on between grant cycles and sustain shared norms. We ask communities to steward RFCs with reference implementations, to host monthly show-and-tell sessions where counterexamples and incidents are discussed candidly, and to grow contributor ladders that welcome new participants. Simple forms of recognition (i.e., pilot-champion badges, release credits) signal the social proof that encourages the next wave of adopters.

Taken together, these roles distribute responsibility without diluting accountability. Funders set incentives; labs harden the theory; builders make it run; auditors verify independently; standards bodies codify the interfaces; and communities sustain the cadence. When each role ships its artifacts on a predictable schedule, verification-by-default ceases to be a slogan and becomes an ecosystem property.

11 Engagement Modes

A movement gains credibility when participation is easy, results are comparable, and artifacts are shared. The engagement modes below are designed to compress the loop from research to enforcement: they create venues where independently developed stacks meet, where evidence is exercised rather than asserted, and where outcomes are recorded in forms that others can reproduce.

Cohort Pilots. Pilots run in 90-day sprints with several organizations adopting the same minimal profile and publishing comparable artifacts. Each cohort begins in **observe** to collect traces, advances to **shadow** to compare proposed decisions with live outcomes, and flips selective clauses to **enforce** once reject budgets are defined. Participants share replay suites that capture typical and adversarial behavior, and they report a small set of public metrics (i.e., policy coverage, certificate verification rate, replay determinism, and incident MTTR) so progress is visible outside the room. At the end of each sprint, teams publish evidence packs that include certificates, replays, provenance, and redactions where necessary, turning lessons learned into portable knowledge.

Interoperability Labs. Interoperability labs are hands-on plug-fests where implementations attempt the same tasks under the same conditions. Over a few focused days, teams cross-verify CERT-V1 certificates, run deterministic egress bake-offs against declared EGRESS-DET-P1 profiles, and exercise revocation and epoch rollover scenarios defined by PERM-UNIFY-R1. Harnesses and oracles are published in advance so time on site is spent diagnosing differences rather than inventing tests. Each lab closes with a public interop report (i.e., what worked, what drifted, and which patches landed) so that the broader community benefits from the convergence work.

Open RFCs. Standards advance through numbered RFCs that are narrow enough to test and stable enough to adopt. Every RFC ships with a reference implementation and a conformance test that an independent party can run without privileged access. Deprecations come with migration guides and timelines so implementers can plan upgrades without halting service. Versioned artifacts, change logs, and an errata window keep the process legible: when an RFC changes, the difference is visible in code, tests, and evidence, not only in prose.

Independent Evaluations. External assessments provide the pressure that keeps guarantees honest. Runtime-verification firms and red teams run standardized testbeds, measure residual risk and coverage, and track drift across releases. Reports include both successes and failures, with counterexamples minimized to actionable traces that others can replay. Funding structures are disclosed to protect independence, and all measurements are tied to public artifacts so results can be reproduced. Over time, a living corpus of accepted counterexamples becomes a shared benchmark that raises the floor for everyone.

Fellowships & FROs. People carry ideas across layers, and sustained work bridges theory and runtime. Fellowships and focused research organizations support contributors who attack the gaps that block deployment, witness compression for field-level IFC, partial zk proofs for streaming prefixes, and formalization of epoch semantics that keep monitors finite. Programs emphasize dual implementations, machine-checked proofs, and runnable predicates, so outputs land as both papers and code. Graduates bring these practices back to labs, platforms, and standards groups, reinforcing the pipeline from concept to conformance.

Together, these modes create momentum: pilots generate traces and instincts; interop labs convert instincts into shared interfaces; RFCs codify those interfaces; independent evaluations test them under pressure; and fellowships keep the hard parts moving. The cadence is repeatable, the artifacts are public, and the field advances in steps that are easy to verify.

12 Current Initiatives

Multiple stacks already compile policies into finite monitors, mediate effects through sidecars, and emit evidence that independent parties can verify. The remaining work is not to invent the paradigm but to scale it and standardize its interfaces so implementations interoperate and auditors can reproduce conclusions without vendor-specific spelunking. The initiatives below describe what is live today, what each track produces, and how we measure progress in the open.

12.1 Provable Runtime Control

We maintain a full pipeline from an action-level specification to enforcement at runtime. Specifications compile to deterministic automata; a sidecar mediates each effect on a labeled alphabet; typed declassification constrains high-to-low releases; and a predicate such as **MonNI** is evaluated online. When **MonNI** accepts every observed prefix and the standard unwinding obligations hold, the local-to-global bridge yields termination-insensitive non-interference, so low views remain equivalent across secret variants. We track whether this system behaves as promised by monitoring p95 latency of the runtime checks, measuring accept/deny precision against the reference semantics, and confirming equality of low views in controlled replays.

12.2 Unified Permissions (Call/Read/Write/Grant)

Agents operate across tools and documents, so authorization must be coherent at that granularity. Our permissions layer uses ABAC/ReBAC guards, evolves through explicit epochs to support safe revocation, and verifies field-level witnesses so membership proofs and bounded taint derivations are checked rather than assumed. With a single decider covering **Call**, **Read**, **Write**, and **Grant**, operators obtain one calculus for both tools and data, and revocation becomes safe and bounded by design. We evaluate the track by the time-to-revocation under load, by the quality and availability of decision explanations suitable for CI and audits, and by the cost of verifying witnesses at runtime.

12.3 Certificates & Provenance

Every emission carries a CERT-V1 record that binds runtime decisions to build-time artifacts. The certificate includes hashes for the governing policy, proofs, automata, and labelers; it records the **MonNI** verdict and the permit decision; and it embeds the results of witness checks. We sign artifacts with DSSE, publish SBOM/SLSA provenance for builds, and optionally log certificate digests to transparency systems. The purpose is straightforward: third parties

should be able to verify what happened without access to implementation internals. Progress is measured by the percentage of certificates independently verified, the latency of inclusion proofs in transparency logs, and the completeness of provenance across releases.

12.4 Deterministic Egress

Reproducible runs require predictable egress. We standardize chunk and flush parameters along with locale and timezone, and we provide optional rate padding when operators need tighter bounds on timing and length channels. A reproducibility harness exercises these profiles so incidents can be replayed byte-for-byte. The desired outcome is twofold: covert channels are bounded, and replay determinism remains high even under load. We therefore report determinism as a percentage under declared profiles, estimate residual capacity in bits per second, and track the false-block rate introduced by padding or shaping.

12.5 Conformance Testing

To keep quality bars portable, we distribute replay kits with both “good” and “bad” traces, adversarial generators that stress chunking and polyglot inputs, and equivalence tests that compare runtime monitors to the reference automata. These materials let independent teams run the same oracles and obtain the same answers. We evaluate the track by coverage of important behaviors, by the ability of the counterexample shrinker to reduce failing traces to minimal, actionable cores, and by the rate at which we detect semantic drift across versions.

12.6 DX Tooling

Developer experience (DX) determines whether enforcement arrives in hours or in weeks. We ship CI gates that verify certificates and replays on every build, editor integrations that render counterexamples in context, scaffolds for adapters, and domain templates for common agent patterns such as redaction-aware RAG, operations automations, and support assistants. The goal is to make *time-to-first-enforcement* a matter of hours. We measure that time-to-enforce directly, track authoring error rates in policies and adapters, and watch policy coverage grow week over week as teams expand what is mediated.

Across these initiatives the emphasis is consistent: each track produces artifacts that others can run, each publishes measurements that invite comparison, and each narrows the distance between what we claim in theory and what we can certify online.

13 Near-term (6–12 months)

The next year is about turning principles into defaults. We will publish narrow standards, exercise them in public events, invite independent pressure tests, seed a marketplace that rewards evidence, and institutionalize a publishing loop so progress is visible outside any single organization.

13.1 Standards Drafts (Public RFCs)

Four drafts anchor the effort. *CERT-V1* specifies the per-emission certificate schema so that runtime decisions bind to build-time artifacts. *EGRESS-DET-P1* fixes the parameters that make replays faithful by standardizing chunking, flushing, and locale/timezone settings. *PERM-UNIFY-R1* defines a unified permission model (with epochs and field-level witnesses) that applies consistently to tools and documents. *MON-NI-BRIDGE* states the preconditions under which local, online acceptance suffices to claim termination-insensitive non-interference globally. Each RFC ships with reference code and a conformance test so adoption requires engineering, not interpretation.

13.2 Interop Events

Quarterly interoperability labs translate prose into practice. Implementations cross-verify CERT-V1 certificates produced by other stacks; deterministic-egress “bake-offs” check that independent runtimes reproduce traces under declared EGRESS-DET-P1 profiles; and revocation and epoch rollover scenarios probe whether PERM-UNIFY-R1 behaves under stress. Results appear on public dashboards that document passes, drifts, and fixes, creating a living record of convergence.

13.3 External Evaluations

Independent scrutiny keeps guarantees honest. Runtime-verification experts and red teams run standardized testbeds and publish coverage and residual-risk reports that quantify accept/deny precision and monitor overhead. Reports include minimized counterexamples and exact environment pins so anyone can reproduce the findings without privileged access.

13.4 Marketplace Alpha

To make verified behavior discoverable and valuable, we launch a small marketplace that lists agents and templates backed by evidence. Entries describe their verified obligations, link to

certificates and replays, and participate in revenue sharing. When dependencies change, SLSA and in-toto provenance triggers automated recertification so that listings remain trustworthy without manual audit fatigue.

13.5 Evidence Loop

Every release should leave a trail others can follow. Teams publish reproducible traces and replays (both “good” and adversarial) alongside sample certificates and the hashes of proofs, automata, and labelers. They include performance envelopes that report latency and throughput under declared profiles, document reject budgets, and write incident post-mortems with minimal counterexamples and accompanying replay bundles. Conformance results record pass/fail outcomes and drift across versions, and inclusion proofs from transparency logs show where certificates live. The loop is simple: ship code, ship evidence, ship measurements.

14 Why This Can Become the Field’s Reference

This program privileges properties that can be enforced online over rhetoric that cannot be checked. By standardizing interfaces (certificates, monitors, replays) rather than whole implementations, it lets many stacks plug in while keeping assurance portable. Supply-chain attestations (SLSA and in-toto) and transparency logs carry provenance directly into runtime evidence, so auditors verify without excavating vendor internals. Deterministic egress makes incidents reproducible; unified permissions make authorization coherent across tools and documents; and the local-to-global bridge explains, in the language of proofs, what online acceptance buys in the large. Because the artifacts are small, testable, and incremental, operators can adopt them clause by clause; because the measurements are public, regulators and platforms can align on evidence instead of promises. Over time, this combination of open profiles, runnable proofs, and repeatable tests yields not just a set of best practices, but a reference that the field can point to with confidence.

15 How to Get Involved: On-ramps & Programs

Building a verifiable AI ecosystem is a participatory act. We invite organizations to join through programs that match their strengths, to begin with a modest technical footprint, and to measure progress in ways that others can independently confirm.

15.1 Programs

A. Research & Standards Working Groups. Groups focused on information-flow control and non-interference, provenance and certificates, deterministic egress, permission epochs, and zk/TEE bridges collaborate to turn repeated practice into testable standards. The expected outputs are concrete: numbered RFCs with clear **MUST/SHOULD** clauses, machine-checked proofs that bind specifications to monitors, reference monitors that others can implement against, formal conformance suites that make results comparable, and position or vision papers that explain trade-offs to the broader community.

B. Engineering & Pilots. Platform teams and product groups translate theory into runtime. Pilot cohorts adopt clause-by-clause rollouts, starting in **observe**, advancing to **shadow**, and flipping to **enforce** once reject budgets are set, and bring their implementations to interoperability labs where decisions are cross-verified. Developer-experience grants target the missing pieces: authoring tools for policies, adapter scaffolds that freeze effect semantics, generators for adversarial tests, and marketplace integrations that surface verified agents.

C. Evaluation & Assurance. Independent evaluators supply pressure and perspective. They run standardized testbeds, verify certificates without privileged access, and, when appropriate, log certificate digests to transparency systems so inclusion proofs are publicly auditable. The artifacts they publish, coverage metrics, residual-risk estimates, and drift reports, become shared baselines that improve with each release.

D. Community & Education. Communities sustain momentum between grant cycles and product launches. Reading groups on runtime verification, IFC, provenance, and zk proofs cultivate shared vocabulary; fellowships fund contributors who bridge theory and runtime; monthly demos surface counterexamples and incident reviews so lessons are institutionalized rather than forgotten.

15.2 Starter Checklist

Getting started does not require a platform rewrite. Run a sidecar in **observe** to export traces and understand your effect surface. Draft a minimal policy, compile it to monitors, and exercise replays until the decisions match intent. Enable **shadow** on low-risk clauses and validate certificates in continuous integration so regressions fail early. Flip targeted rules to **enforce** once a reject budget is agreed, then publish evidence packs, certificates, replays, provenance, for others to verify. Finally, join a working group and either propose an RFC or implement one; both contributions move the ecosystem forward.

15.3 Ecosystem KPIs

We track progress with a small set of comparable indicators. Coverage reports the percentage of effects that are labeled and monitored. Certificate verification rate measures how often independent verifiers validate emissions successfully. Replay determinism quantifies how faithfully runs reproduce under declared profiles. Reject-budget adherence shows whether enforcement remains within operational guardrails, and performance overhead reports p95 monitor latency and end-to-end throughput impacts. At the ecosystem level, we count verified agents and templates, the number of interoperability events and independent evaluations, the extent of platform integrations such as deployment gates, the presence of regulated deployments, and the publication of incident reports accompanied by replays.

15.4 Code of Practice

The social contract is simple: value evidence over rhetoric; publish artifacts that others can run; collaborate respectfully across institutions. Every claim carries a clear scope statement that distinguishes what is guaranteed from what remains out of scope. Governance is multi-stakeholder and transparent, with numbered RFCs, public roadmaps, and explicit deprecation and migration policies so adopters can plan with confidence.

Current & Potential Participants

This effort is already multi-institutional, and the door remains open. As conveners and funders, organizations such as Convergent Research, which operates Focused Research Organizations that bridge proof-of-concept to public infrastructure, and DARPA, with its long track record of catalyzing cyber and assurance technologies, are natural anchors. In the public sector, the United States has established the U.S. AI Safety Institute at NIST (now operating via the Center for AI Standards and Innovation and the AI Safety Institute Consortium), while the United Kingdom has stood up the AI Security Institute; both are explicitly focused on evaluation science, testbeds, and standardization pathways that will shape verification-by-default practices. Foundations and member-driven coalitions around software assurance, including the Open Source Security Foundation (OpenSSF), further convene industry and government to align on practical supply-chain hardening. The initial list we cited (i.e., Beneficial AI, Convergent Research, DARPA, and the AGI Social Contract) fits within this broader landscape of funders who can underwrite pilots, interop events, and the development of open evidence packs.

On the standards and policy front, several bodies are already producing directly relevant,

testable artifacts. NIST’s AI Risk Management Framework provides a risk vocabulary, governance scaffolding, and measurement-oriented guidance that complements runtime guarantees. ISO/IEC 42001 establishes an AI management system standard that platforms can certify against while retaining room for technological diversity. For provenance and transparency, the W3C PROV family offers a mature data model and ontology to represent the who/what/when of artifacts and emissions, while IETF’s SCITT working group is defining integrity, transparency, and trust building blocks for supply chains. In parallel, supply-chain transparency in software is maturing through SBOM initiatives led by CISA, with SPDX (Linux Foundation/ISO standard) and OWASP CycloneDX providing machine-readable interchanges. These efforts are not merely adjacent: they are the policy and interchange substrates into which runtime certificates and replay kits should neatly slot.

The open-source infrastructure that makes evidence practical is also advancing rapidly. The Sigstore stack (including the Rekor transparency log) has normalized public, append-only logging for software attestations, while in-toto provides a signed, end-to-end framework to assert and verify each step of a build pipeline. For authorization and operational policy, Open Policy Agent (OPA) has emerged as a widely adopted, CNCF-graduated decision engine; it is a useful baseline for request-scoped authorization even when event-level information-flow enforcement calls for monitors over labeled alphabets. Remote attestation and confidential execution are likewise maturing: production TEEs and their attestation flows (e.g., AMD SEV-SNP, Intel TDX, ARM’s confidential compute architectures, and AWS Nitro Enclaves) enable operators to attest how code ran when “what it emitted” is not sufficient. In parallel, zero-knowledge proof systems and zkVMs (e.g., RISC Zero; STARK-based systems from StarkWare; and ecosystems around Halo2 or zkSync) offer cryptographic attestations for compute steps where TEEs are unsuitable or where verifiers demand hardware-minimal trust.

Independent evaluators and formal-methods practitioners form a complementary backbone for verification at adoption time. Organizations such as METR (the Model Evaluation and Threat Research group) run red-team style evaluations and publish threat-informed reports; formal-methods engineering shops including Runtime Verification, Galois, and Certora build proofs, monitors, and counterexample tooling that translate specifications into deployable assurance. Beyond professional evaluators, the runtime-verification research community and standards-minded industry labs (including those participating in OpenSSF working groups or IETF SCITT) are publishing reference monitors, conformance oracles, and open testbeds that others can replay and extend.

Finally, the builder ecosystem is broadening. In addition to the projects and companies already enumerated in earlier drafts, Atlas Computing, Project VAIL, Lunal, Phala Network, and Harmonic, the verification substrate increasingly includes maintainers of transparency

logs and verifiers, SBOM producers and consumers, policy-engine vendors, TEE attestation services, and zk-proof providers. As the ecosystem coalesces around reproducible certificates, deterministic egress, and unified permissions, we expect major platforms to integrate certificate gates by default and to publish sector-specific reference profiles that reflect real regulatory obligations.

This list is intentionally porous and will evolve. New participants are encouraged to propose pilots, contribute reference monitors and proof artifacts, and publish evidence packs that third parties can independently verify. The north star remains constant: evidence over rhetoric, with specifications, monitors, attestations, and replays that others can check without asking anyone to “just trust us.”

Appendix A: Glossary

This glossary fixes terminology used throughout the paper so that implementers, auditors, and policymakers share the same referents. Definitions are intentionally concise and operational.

NI *Non-interference*. The global confidentiality claim that low-view projections of traces remain equivalent across secret variants; in practice, it is the property that well-typed declassifications and complete mediation preserve indistinguishability.

TI *Termination-insensitive NI*. A pragmatic variant of NI that abstracts away termination behavior, enabling an online bridge from local monitoring predicates to a global claim.

MonNI

Monitoring predicate for NI. An online, prefix-wise condition sufficient to imply NI under complete mediation and typed declassification; evaluated by the runtime over the labeled alphabet.

Deterministic Egress

A declared profile (fixed chunk size, flush cadence, locale, timezone) that bounds timing/length channels and makes replays faithful to original runs.

Reject Budget

An agreed allowance of denials per period that preserves business continuity while tightening enforcement; paired with clause-level rollout (**observe**→**shadow**→**enforce**).

Appendix B: DFA JSON (Spec $\rightarrow$ Monitor)

The example below illustrates how an action-level specification compiles to a deterministic finite automaton over a labeled alphabet. Each transition encodes an obligation (e.g., quotas, taint rules); metadata binds the automaton back to the source policy and proof artifacts.

```
{
  "alphabet": ["tool.call", "file.read", "file.write", "net.send", "declass"],
  "states": ["s0", "s1", "s2"],
  "start": "s0",
  "accepting": ["s0", "s1", "s2"],
  "transitions": [
    {"from": "s0", "on": "tool.call:name=email.draft", "to": "s1"},
    {"from": "s1", "on": "quota(email.draft)<=10/5min", "to": "s1"},
    {"from": "s*", "on": "net.send:taint>=secret && !has(declass.redact)", "to": "REJECT"}
  ],
  "metadata": {
    "policy_hash": "sha256:...",
    "proof_hash": "sha256:...",
    "compiler_version": "0.3.2"
  }
}
```

Appendix C: CERT-V1 Example

A per-emission certificate ties runtime verdicts to build-time artifacts and optional attestations. The schema below is representative; real systems should pin versions and include signatures and transparency references where applicable.

```
{
  "schema": "cert-v1",
  "bundle_id": "pab-2025.09.01+build17",
  "sidecar_build": "1.3.0",
  "epoch": 42,
  "event": "net.send",
  "args_fingerprint": "sha256:...",
  "policy_hash": "sha256:...",
  "proof_hash": "sha256:..."
}
```

```

"automata_hash": "sha256:...",
"labeler_hash": "sha256:...",
"ni_monitor": "accept",
"permit_decision": "accept",
"path_witness_ok": true,
"label_derivation_ok": true,
"attestation": {
  "type": "tee",
  "quote_ref": "https://attest.vendor/quotes/abc123"
},
"transparency_ref": "rekor:logindex/123456",
"signatures": [
  {"alg": "ed25519", "sig": "base64:..."}
]
}

```

Appendix D: Replay Bundle Format

Replay bundles make evidence executable: a verifier rehydrates environment pins, replays traces under the declared egress profile, and compares outcomes against oracles. This enables byte-for-byte incident diagnosis and objective conformance checks.

```

{
  "profile": "EGRESS-DET-P1",
  "env": {"locale": "en_US", "timezone": "UTC", "chunk_bytes": 2048, "flush_ms":
    50},
  "fixtures": {"corpus": "s3://.../demo-corpus.tar.zst"},
  "traces": [
    {"id": "trace-001", "events": "...", "cert_refs": ["..."]},
    {"id": "trace-002", "events": "...", "cert_refs": ["..."]}
  ],
  "oracles": {"pass": ["trace-001"], "fail": ["trace-002"]}
}

```

Appendix E: Governance: RFC Lifecycle

Standards evolve through a predictable lifecycle so implementers can plan upgrades without breaking service. Each stage produces public artifacts, code, tests, and migration notes, that make changes legible.

1. **Draft.** A narrowly scoped problem statement, explicit **MUST/SHOULD** clauses, a reference implementation, and a conformance test.
2. **Pilot.** Two independent implementations pass the conformance suite; interop notes record any ambiguities and fixes.
3. **Standard.** Broad adoption across stacks; public interop reports; versioned deprecations with timelines.
4. **Maintenance.** Security advisories, migration guides, and transparency-log entries that trace deprecations and replacements.

Appendix F: Conformance Suite Skeleton

A minimal but complete conformance layout keeps tests reproducible and extensible. Profiles, traces, and oracles live beside verifiers so third parties can run them unmodified.

```
tests/  
  egress_det_p1/  
    profiles/*.json  
    traces/good/*.trace  
    traces/bad/*.trace  
    oracle.yaml  
  cert_v1/  
    samples/*.json  
    verifier/*.py  
  perm_unify_r1/  
    traces/*.trace  
    witnesses/*.json
```

Appendix G: SLO Targets

Service-level objectives make promises measurable and regressions actionable. Targets below are defaults; sector profiles may tighten them.

SLO	Target
Certificate verification rate	99.99% per release window
Replay determinism	99.9% under EGRESS-DET-P1
Monitor overhead	p95 < 2 ms/event
Reject budget adherence	$\geq 99\%$ of periods
Inclusion-proof latency	p95 < 200 ms

Appendix H: Lean Skeleton (Semantics, MonNI, and Obligations)

This skeleton shows the core objects and theorem statements. The full project (sources, pinned toolchain, olean caches, and generated docs) is distributed with releases; its SHA256 appears as `proof_hash` in CERT-V1.

```

/-! # Core objects (Lean 4) -/

namespace VerAI

/-- Observable events over which monitors run. -/
inductive Event
| toolCall (name : String) (args : Std.HashMap String String)
| fileRead (path : String)
| fileWrite (path : String)
| netSend (bytes : ByteArray)
| declass (site : String) -- typed declassification site
deriving DecidableEq, Repr

abbrev Trace := List Event

/-- Labels and low-view projection (erasing high fields). -/
structure Label := (isHigh : Bool)
def lowErase (e : Event) : Option Event := -- consult label map + declass rules
  some e -- (placeholder; implemented in repo)

def lowView : Trace → Trace
| [] => []

```

```

| e :: t => match lowErase e with
  | some e' => e' :: lowView t
  | none => lowView t

/-- Deployment invariants (frozen adapters and typed declassification). -/
def Mediated (t : Trace) : Prop := True -- every effect passes adapters
def DeclassOK (e : Event) : Prop := True -- site justified by policy

/-- Prefixes of a trace (including empty). -/
def prefixes : Trace List Trace
| [] => [[]]
| e :: t => let ps := prefixes t; [] :: (ps.map (List.cons e))

/-- Online monitor predicate sufficient for TI-NI under mediation. -/
def MonNI (t : Trace) : Prop :=
  p prefixes t, ( e p, DeclassOK e ) /\ Mediated p

/-- Low-view equality for termination-insensitive non-interference. -/
def lowEq (t t : Trace) : Prop := lowView t = lowView t

/-- ActionDSL fragment and DFA compilation (signatures only here). -/
structure Spec := (stub : Unit) -- concrete grammar in repo
structure DFA := ( : Type) (q : Type) (start : q)
               (accept : q Prop) ( : q q)

def compile (S : Spec) : DFA := by
  admit -- implemented in repository; JSON encoder binds to automata_hash

def LangDFA (A : DFA) : Set (List A.) := {w | True} -- placeholder
def Denote (S : Spec) : Set (List Event) := {w | True} -- placeholder

/-- H.1 Compilation soundness: L(DFA) = spec for supported fragment. -/
theorem compiler_sound (S : Spec) :
  w, w LangDFA (compile S) w Denote S := by
  admit

/-- H.2 Local-to-global confidentiality (TI-NI) under mediation + declass. -/
theorem MonNI_implies_TINI

```

```

(t t : Trace)
(Hmed : Mediated t) (Hmed : Mediated t)
(Hmon : MonNI t) (Hmon : MonNI t)
(HsecretVar : True) : -- differ only in high parts; formalized in repo
lowEq t t := by
  admit

/-- H.3 Replay faithfulness at low view under EGRESS-DET-P1. -/
structure EgressProfile := (chunk : Nat) (flushMs : Nat)
                           (locale : String) (tz : String)

def replay (p : EgressProfile) (t : Trace) : Trace := t -- idealized

theorem replay_low_view_equiv
  (p : EgressProfile) (t : Trace)
  (Hp : True) : -- P1 constraints; checked in implementation
  lowView (replay p t) = lowView t := by
  admit

end VerAI

```

Build & CI notes. Releases pin the Lean toolchain and dependencies; CI builds proofs, exports olean caches, and emits the `proof_hash`. The DFA JSON emitted by the compiler is hashed as `automata_hash`. Both hashes appear in CERT-V1 so auditors can tie runtime evidence to specific theorems and monitors.